NutriCompass - System Architecture Suggestions

for the following system, suggest suitable architectural styles/patterns to use.

NutriCompass provides users with structured information about dietary supplements and their documented relationships through AI-powered analysis of scientific literature and supplement databases. Users interact with the system by scanning nutrition facts or ingredient labels from supplement packaging using their device camera, typing supplement names directly, or searching from a comprehensive pre-loaded database of supplements. The AI processes scanned label images through optical character recognition and ingredient extraction, identifying active compounds, dosages, and formulation details. Users can search for supplements by brand name (e.g., "Nature Made Vitamin D3") or general ingredient name (e.g., "vitamin D," "magnesium"), with the system presenting matches from both the pre-loaded database and previously scanned items stored in the user's personal library. Once supplements are selected—whether newly scanned, previously stored, or chosen from the database—the system analyzes their potential interactions and provides evidence-based information about their combined effects. All user-scanned data is persistently stored within the app, building a personal supplement library that can be referenced for ongoing analysis. The pre-loaded database contains extensive information on thousands of supplements, enabling users to perform comparative analysis between multiple database supplements, compare scanned supplements against database entries, or analyze combinations of both to understand documented interactions, absorption patterns, and reported effects.

The system functions strictly as an AI-driven decision support tool, not a decision-making agent. It operates within clearly defined boundaries where AI is used for image processing, ingredient recognition, information retrieval, evidence aggregation, and interaction analysis, but explicitly not for making judgments about individual safety, supplement appropriateness, dosing guidance, or disease treatment claims. When users select multiple supplements for analysis, the AI processes data to surface documented interaction patterns and synthesize findings from multiple sources, presenting them as "reported cases exist," "observed in research," or "may affect absorption or metabolism," but does not interpret what those findings mean for any individual user. Rather than attempting to resolve scientific uncertainty through AI inference, the system uses AI to identify and categorize uncertainty itself—distinguishing evidence levels, flagging conflicting findings, and revealing gaps in available data. The AI's role is strictly confined to organizing and structuring information from both user-scanned supplements and the comprehensive database, supporting user decision-making while keeping all interpretive judgment, risk assessment, and action decisions entirely with the user.

Architecturally, NutriCompass is designed as a data-driven, AI-powered evidence aggregation system with hard conceptual boundaries enforced throughout its information pipeline, data processing logic, and AI model behavior. The system's design philosophy—"decision support, not decision making"—permeates every layer from image capture and OCR processing to database search and interaction analysis to interface presentation. The data architecture maintains three primary information sources: a comprehensive pre-loaded supplement database with standardized ingredient profiles and interaction data, a user-specific library of scanned supplements with extracted ingredients and metadata, and a scientific evidence base containing documented interactions and research findings. AI models are trained and constrained to extract factual ingredient information from label images without generating dosing recommendations, to match user queries against database entries without inferring suitability, to identify evidence patterns from literature without generating personalized advice, and to structure comparative analyses

without embedding judgment rules. The database search and matching algorithms prioritize accurate ingredient identification and interaction detection while maintaining clear separation between observed evidence (extracted from sources), evidence metadata (strength, source type, consistency), and user context (supplement combinations being queried). The user interface flow—from scan/search to selection to analysis—is designed to reinforce information presentation rather than recommendation generation, with all outputs framed as documented evidence rather than guidance. This design positions AI as a powerful information processing engine operating within carefully designed rails, testing the hypothesis that clear upfront architectural constraints allow AI to enhance system capabilities through label scanning, database search, and interaction analysis while maintaining the fundamental boundary that keeps decision-making authority with the user.

Below is a **clean, defensible set of architectural styles and patterns** that fit NutriCompass *as you described it*, with explicit alignment to your core principle:

> **AI as decision support, never decision-making**

I'll structure this from **top-level architecture** → **core data/AI pipeline** → **enforcement of boundaries**, and I'll be blunt where certain patterns are *necessary* rather than optional.

---

1. Overall System Architecture

Layered Architecture (Strict, Not Relaxed)

Why it fits NutriCompass

- Your system depends on **hard conceptual boundaries** (capture → extraction → evidence → presentation).
- You *must* prevent AI outputs from leaking into recommendation logic.
- Layering allows you to enforce **what information is allowed to pass upward**.

Recommended layers

1. **Presentation Layer**
    - UI framing, evidence labels, disclaimers
    - Zero inference, zero transformation of meaning
2. **Application / Orchestration Layer**
    - Coordinates scan/search/compare flows
    - Explicitly *non-intelligent*
3. **AI Processing Layer**
    - OCR
    - Ingredient extraction
    - Evidence clustering
    - No user context awareness beyond selected items

4. **Domain Logic Layer**
    - Supplement entities
    - Interaction entities
    - Evidence metadata (strength, conflict, source)
5. **Data Layer**
    - Preloaded database
    - User library
    - Evidence corpus

Key rule

> AI never crosses directly into Presentation or Domain Logic.

This is critical for regulatory defensibility.

---

2. Core Interaction Model

Pipes-and-Filters Architecture (for AI Pipelines)

This is one of the *strongest matches* to your philosophy.

Pipeline example

```
Image Capture
 → OCR Filter
 → Ingredient Extraction Filter
 → Normalization Filter
 → Evidence Retrieval Filter
 → Evidence Classification Filter
 → Presentation Formatter
```

Why this matters

- Each stage has **narrow responsibility**
- You can audit and constrain each filter independently
- Prevents “creative drift” common in monolithic AI inference

Enforcement advantage

- You can formally state:
  “No filter is allowed to output recommendations, probabilities, or prescriptions.”

---

3. Data Architecture

Data-Centric / Repository Pattern

NutriCompass is fundamentally **data-driven**, not behavior-driven.

Repositories

- `SupplementRepository` (pre-loaded database)
- `UserSupplementRepository` (scanned items)
- `EvidenceRepository` (studies, reports, cases)

Why

- Centralized, queryable data sources
- Clean separation between:
  - **Observed facts**
  - **Evidence metadata**
  - **User selections**

This avoids the classic trap of "smart services with embedded judgment."

---

4. Evidence Handling & Uncertainty

Knowledge Graph + Evidence Annotation Pattern

You are *not* just storing interactions—you are storing **relationships with uncertainty**.

Model

- Nodes: Ingredients, Compounds, Supplements
- Edges: Interaction types (absorption, metabolism, antagonism)
- Edge metadata:
  - Evidence strength
  - Source type
  - Consistency/conflict flags

Why this is essential

- Enables:
  - "Reported cases exist"
  - "Observed in research"
  - "Conflicting evidence"
- Avoids collapsing uncertainty into a score or recommendation

This directly supports your claim:

> "AI identifies uncertainty, not resolves it."

---

5. Search & Matching

CQRS (Command Query Responsibility Segregation)

Commands

- Scan supplement
- Store scanned item
- Add to user library

Queries

- Search supplement database
- Compare supplements
- Retrieve interaction evidence

Why CQRS fits

- Scanning *changes state*
- Analysis *never changes state*
- Prevents analysis logic from mutating user data or preferences

This reinforces **decision support only**.

---

6. AI Integration Style

AI-as-a-Service (Constrained Microservice or Module)

Important distinction

- Not “AI everywhere”
- AI is a **replaceable, auditable subsystem**

Characteristics

- Stateless where possible
- Input-output contracts:
  - Input: images, ingredient strings, IDs
  - Output: structured facts only
- No access to:
  - User health conditions
  - Preferences
  - Prior decisions

This makes your system resilient to:

- Model upgrades
- Regulatory review
- Model replacement

---

7. Boundary Enforcement

Policy-Enforced Architecture (Hard Rails)

This is not optional for your system.

Examples

- Output schemas disallow:
  - “should”
  - “recommended”
  - “safe/unsafe”
- Evidence labels are enumerated, not generated
- UI templates constrain phrasing

Architectural role

- Policies act as *compile-time and runtime guards*
- AI outputs are validated before rendering

This is how you prove:

> “The system cannot accidentally become advisory.”

---

8. UI & User Flow

Information-Centric Presentation Pattern

Instead of:

- Wizard-based recommendations
  Use:
- Evidence cards
- Comparison tables
- Source-linked summaries

Why

- Keeps cognitive authority with the user
- Aligns UX with system intent

- Avoids dark patterns of implied guidance

---

9. Optional but Strong Enhancers

Event-Driven Architecture (Internally)

- Scan completed
- Evidence updated
- Database refreshed

Useful for scalability, but **not required initially**.

Final Architectural Stack (Concise)

Concern	Pattern
Overall structure	**Strict Layered Architecture**
AI processing	**Pipes-and-Filters**
Data management	**Repository + Data-Centric**
Evidence modeling	**Knowledge Graph + Evidence Annotation**
State vs analysis	**CQRS**
AI integration	**Constrained AI-as-a-Service**
Safety boundaries	**Policy-Enforced Architecture**
UX philosophy	**Information-Centric Presentation**

Coach's Bottom Line (Direct)

You've described a system that **fails catastrophically** if architectural boundaries blur.
These patterns are not just "good practice"—they are what makes your *research hypothesis credible*:

> *That AI can dramatically enhance capability **without** assuming authority.*

If you want, next we can:

- Map these patterns directly to **Swift data models**
- Or turn this into a **formal Architecture Design section** suitable for an academic paper

define each layer with composing components.

Below is a **precise, implementation-oriented definition of each architectural layer**, with **explicit composing components** and **clear responsibility boundaries**.
I'll stay aligned with your stated philosophy and avoid introducing any component that could accidentally imply recommendation logic.

NutriCompass – Layered Architecture with Composing Components

1. Presentation Layer (UI & Information Framing)

Primary responsibility
Render structured information, evidence, and uncertainty **without interpretation, guidance, or inference**.

Key rule
This layer **never transforms meaning**—it only formats and displays.

Composing Components

- **Scan & Input Views**
  - Camera capture UI
  - Manual text input (brand / ingredient search)
- **Search & Selection Views**
  - Database supplement list
  - User library list (scanned items)
  - Multi-selection controls
- **Analysis Result Views**
  - Interaction summary panels
  - Evidence cards (per interaction)
  - Comparison tables
- **Evidence Framing Components**
  - Evidence level badges (e.g., "reported cases," "observed in research")
  - Conflict flags
  - Source attribution links
- **UI Policy Enforcers**
  - Output phrase templates (enumerated, non-generative)
  - Visual emphasis rules (evidence > conclusion)
- **Disclaimer & Boundary Indicators**
  - "Information only" banners
  - "No personalized advice" indicators

2. Application / Orchestration Layer

Primary responsibility
Coordinate user flows and pipeline execution **without embedding intelligence**.

Key rule
This layer knows *what happens next*, not *what it means*.

Composing Components

- **Workflow Controllers**
  - Scan workflow coordinator
  - Search workflow coordinator
  - Comparison / analysis workflow coordinator
- **Request Routers**
  - Route inputs to OCR, search, or analysis pipelines
- **Session State Manager**
  - Tracks selected supplements
  - Maintains temporary analysis context
- **Permission & Consent Manager**
  - Manages user consent for storage
  - Enforces local-only or cloud-allowed paths
- **Result Assembly Coordinator**
  - Collects outputs from AI and data layers
  - Assembles structured result objects (no interpretation)

---

3. AI Processing Layer (Constrained Intelligence Layer)

Primary responsibility
Extract, normalize, and structure factual information.

Key rule
AI outputs **facts and classifications only**, never judgments.

Composing Components

- **Image OCR Module**
  - Label text extraction
  - Confidence scoring (non-semantic)
- **Ingredient Extraction Module**
  - Active compound identification

    - Dosage text extraction (verbatim)
- **Ingredient Normalization Module**
    - Synonym resolution
    - Canonical ingredient mapping
- **Literature Retrieval Module**
    - Query scientific databases
    - Retrieve relevant publications
- **Evidence Pattern Detection Module**
    - Detects interaction mentions
    - Groups findings by interaction type
- **Evidence Classification Module**
    - Tags evidence strength
    - Flags conflicts and inconsistencies
- **Output Schema Validator**
    - Ensures AI outputs conform to allowed structures
    - Rejects prohibited language or inference

---

4. Domain Logic Layer (Non-AI, Rule-Bound)

Primary responsibility
Define the **meaning of entities and relationships** without interpretation.

Key rule
This layer encodes **what exists**, not *what should be done*.

Composing Components

- **Domain Models**
    - Supplement
    - Ingredient / Compound
    - Interaction
    - EvidenceRecord
- **Interaction Taxonomy**
    - Absorption
    - Metabolism
    - Synergistic / Antagonistic
- **Evidence Metadata Model**
    - Source type (clinical, observational, in vitro)

    - Strength level
    - Consistency indicator
- **Comparison Engine**
    - Aligns ingredients across supplements
    - Matches interactions without scoring or ranking
- **Constraint Rules**
    - Separation of observed data vs metadata
    - No user health attributes allowed

---

5. Data Layer

Primary responsibility
Persist, retrieve, and version data **without logic or inference**.

Key rule
Data stores facts; meaning is added *above* this layer.

Composing Components

- **Pre-Loaded Supplement Database**
    - Standardized supplement profiles
    - Ingredient compositions
- **User Supplement Library**
    - Scanned supplement records
    - Extracted ingredient metadata
- **Scientific Evidence Store**
    - Publications
    - Reported interaction cases
- **Repository Interfaces**
    - SupplementRepository
    - UserSupplementRepository
    - EvidenceRepository
- **Indexing & Search Structures**
    - Ingredient lookup indices
    - Brand and synonym indices
- **Versioning & Provenance Manager**
    - Source tracking
    - Data update history

---

6. Policy & Boundary Enforcement Layer (Cross-Cutting)

Primary responsibility
Ensure the system **cannot cross from decision support into decision making**.

Key rule
This layer overrides everything else if violated.

Composing Components

- **Language Policy Engine**
  - Prohibited terms list
  - Allowed phrasing enumerations
- **Schema Enforcement Engine**
  - Validates AI and domain outputs
  - Rejects recommendation-like structures
- **Context Firewall**
  - Prevents AI from accessing user health data
- **Audit & Logging Module**
  - Tracks AI inputs and outputs
  - Enables post-hoc inspection
- **Regulatory Mode Switch**
  - Adjusts strictness for jurisdictional requirements

---

7. Infrastructure / Platform Layer

Primary responsibility
Provide execution, performance, and isolation guarantees.

Composing Components

- **On-Device ML Runtime**
  - OCR models
  - Ingredient extraction models
- **Secure Storage**
  - Encrypted local databases
- **Optional Cloud Services**
  - Literature updates
  - Database refresh
- **Model Version Manager**

- Ensures reproducibility
- Supports rollback

Responsibility Boundary Summary (Critical)

Layer	Allowed
Presentation	Display, frame, label
Application	Coordinate, route
AI	Extract, classify, organize
Domain	Define entities, relations
Data	Store, retrieve
Policy	Block, validate, audit

❌ **No layer is allowed to infer suitability, safety, or action.**

Coach's Closing Note (Direct)

What you've defined is **not just an app architecture**—it's an *argument made executable*.
Each layer exists to **prove** that AI can be powerful *without being authoritative*.

If you want next:

- I can turn this into a **formal UML-level component diagram**
- Or map it directly into **Swift module boundaries for NutriCompass**

> establish dependencies among components between layers.

Below is a **clear, enforceable dependency structure** for NutriCompass.
I'll define **who can depend on whom, in which direction, and why**, and I'll explicitly mark **forbidden dependencies**—because those are just as important for your system's credibility.

I'll start with **layer-level dependency rules**, then go **component-to-component**.

1. Global Dependency Rules (Non-Negotiable)

Allowed dependency direction (top → bottom only)

```powershell

Presentation
  ↓
Application / Orchestration
  ↓
Domain Logic
```

```
  ↓
AI Processing
  ↓
Data
```

Cross-cutting

```scss
Policy & Boundary Enforcement
↳ may observe & validate all layers
↳ may block upward data flow
↳ may not be bypassed
```

Explicitly forbidden

- ❌ AI → Presentation
- ❌ AI → Application decisions
- ❌ AI → User context
- ❌ Data → Domain logic decisions
- ❌ Presentation → AI direct calls

This keeps **authority flowing downward** and **information flowing upward only after validation**.

---

2. Layer-to-Layer Dependency Map

2.1 Presentation Layer Dependencies

Depends on

- **Application / Orchestration**
  - Receives view models
  - Triggers workflows
- **Policy Layer**
  - Validated phrasing
  - Allowed UI templates

Does NOT depend on

- AI Processing ❌
- Data repositories ❌
- Domain models directly ❌

Reason

Presentation must never:

- Trigger inference
- Access raw evidence
- Bypass framing constraints

2.2 Application / Orchestration Layer Dependencies

Depends on

- **Domain Logic**
  - Entity definitions
  - Interaction taxonomy
- **AI Processing**
  - OCR execution
  - Evidence extraction pipelines
- **Policy Layer**
  - Output validation
  - Flow constraints

Does NOT depend on

- Data storage implementations ❌ (only repositories)
- AI internals ❌ (only interfaces)

Reason

This layer is a **traffic controller**, not a decision-maker.

2.3 Domain Logic Layer Dependencies

Depends on

- **Data Layer (via repositories only)**
  - Fetch supplements
  - Fetch evidence records
- **Policy Layer**
  - Entity integrity constraints

Does NOT depend on

- AI Processing ❌
- Presentation ❌

- Application workflows ❌

Reason

Domain logic must remain:

- Deterministic
- Inspectable
- Non-probabilistic

2.4 AI Processing Layer Dependencies

Depends on

- **Data Layer**
  - Evidence corpus
  - Ingredient dictionaries
- **Policy Layer**
  - Output schemas
  - Language constraints

Does NOT depend on

- Domain rules ❌
- User context ❌
- Presentation ❌

Reason

AI should **see facts, not meaning** and **structure data, not interpret it**.

2.5 Data Layer Dependencies

Depends on

- **Infrastructure only**
  - Storage engines
  - Indexing mechanisms

Does NOT depend on

- AI ❌
- Domain ❌

- Application ❌
- Presentation ❌

Reason

Data must remain:

- Passive
- Reusable
- Non-opinionated

2.6 Policy & Boundary Enforcement Layer Dependencies

Depends on

- Nothing below it (conceptually)

Observes & validates

- AI outputs
- Domain objects
- Application payloads
- Presentation-bound view models

Can block

- Upward data propagation
- UI rendering
- Storage writes

Reason

This layer is your **constitutional court**.

---

3. Component-Level Dependency Matrix

3.1 Scan Workflow (Example)

```markdown

CameraView
 → ScanController
   → OCRService
     → RawText
   → IngredientExtractionService
     → ExtractedIngredients
   → PolicyValidator
```

```
      → ValidatedIngredientSet
    → UserSupplementRepository
```

Forbidden here

- OCRService → UserLibrary ❌
- IngredientExtraction → UI ❌

3.2 Search Workflow

```nginx
SearchView
 → SearchController
   → SupplementRepository
   → UserSupplementRepository
   → DomainMatcher
   → PolicyValidator
   → SearchResultViewModel
```

Forbidden

- Repository → UI ❌
- DomainMatcher → AI ❌

3.3 Interaction Analysis Workflow

```markdown
AnalysisView
 → AnalysisController
   → DomainInteractionEngine
     → EvidenceRepository
     → InteractionTaxonomy
   → EvidenceAggregationService (AI)
     → EvidencePatterns
   → PolicyValidator
   → AnalysisResultViewModel
```

Forbidden

- EvidenceAggregation → Interpretation ❌
- DomainInteractionEngine → Recommendation ❌

---

4. Dependency Table (Condensed)

Component	Allowed Dependencies	Forbidden Dependencies
UI Views	Application, Policy	AI, Data
Controllers	Domain, AI, Policy	Data internals

Component	Allowed Dependencies	Forbidden Dependencies
Domain Models	Data (repos)	AI, UI
AI Modules	Data, Policy	User context
Repositories	Storage	AI, UI
Policy Engine	All (observe)	Bypass

---

5. Why This Matters (Bluntly)

If NutriCompass ever faces:

- Reviewer scrutiny
- IRB-style ethical review
- Regulatory questions
- Academic replication

This dependency structure lets you **prove**, not claim:

> "The system cannot make decisions even if it wanted to."

That is a **structural guarantee**, not a documentation promise.

Next logical steps (you choose)

1. Convert this into a **formal component diagram (UML / C4)**
2. Map directly to **Swift module boundaries**
3. Extract a **paper-ready 'Architectural Safeguards' section**

Say which one you want next.

> identify use cases from the user perspective based on the system description.

Below are **user-perspective use cases** derived strictly from your system description.
They are written in **user language**, not system language, and they deliberately avoid any implication of recommendation or decision-making.

I'll group them by **user intent**, then give each use case a **clear scope and boundary**.

NutriCompass – User-Centered Use Cases

UC-1. Scan a Supplement Label

User goal
Capture supplement information from physical packaging.

Description
The user uses the device camera to scan a supplement's nutrition facts or ingredient label. The system extracts visible text and identifies listed ingredients, dosages (as printed), and formulation details.

System boundary

- Extracts what is printed
- Does not interpret dosage meaning
- Does not assess safety or suitability

Outcome

- A structured supplement entry is created
- The supplement can be stored in the user's personal library

---

UC-2. Search for a Supplement by Name or Ingredient

User goal
Find supplement information without scanning a label.

Description
The user searches by brand name (e.g., product name) or by general ingredient name (e.g., "magnesium"). The system retrieves matching entries from the pre-loaded supplement database and the user's personal library.

System boundary

- Returns matches based on name and ingredient similarity
- Does not rank by quality, effectiveness, or appropriateness

Outcome

- Search results list database supplements and previously scanned items

---

UC-3. View Supplement Details

User goal
Understand what a supplement contains.

Description
The user selects a supplement (scanned or database entry) to view its ingredient list, extracted compounds, formulation details, and source provenance.

System boundary

- Displays factual content only
- Does not explain what the supplement "does" for the user

Outcome

- User gains structured, source-linked information about a supplement

---

UC-4. Build a Personal Supplement Library

User goal
Maintain a persistent record of supplements they use or are interested in.

Description
The user saves scanned supplements or selected database supplements into their personal library for future reference and analysis.

System boundary

- Storage is user-controlled
- No inference or personalization is applied

Outcome

- A reusable personal supplement collection is created

UC-5. Select Multiple Supplements for Analysis

User goal
Explore documented relationships between supplements.

Description
The user selects two or more supplements—any combination of scanned items and database entries—to analyze their documented interactions.

System boundary

- Analysis is limited to reported evidence
- No safety conclusions or recommendations are generated

Outcome

- A defined set of supplements is prepared for interaction analysis

---

UC-6. View Documented Supplement Interactions

User goal
See what has been reported about supplement combinations.

Description
The system presents documented interaction patterns between selected supplements, such as absorption effects, metabolic interactions, or reported antagonistic/synergistic observations.

System boundary

- Outputs are framed as:
  - “Reported cases exist”
  - “Observed in research”
  - “Evidence is conflicting”
- No interpretation of risk or benefit

Outcome

- The user receives structured evidence summaries with sources

---

UC-7. Compare Supplements

User goal
Understand differences and overlaps between supplements.

Description
The user compares multiple supplements to view differences in ingredients, formulations, and documented interactions.

System boundary

- Comparison is factual and tabular
- No “better than” or “recommended” language

Outcome

- Side-by-side comparison of supplement attributes and evidence

---

UC-8. Review Evidence Sources and Strength

User goal
Assess how strong or consistent the evidence is.

Description
The user inspects the evidence underlying reported interactions, including source types, consistency indicators, and conflicting findings.

System boundary

- Evidence strength is categorized, not scored
- The system does not resolve uncertainty

Outcome

- User gains transparency into the scientific basis of the information

---

UC-9. Identify Gaps or Uncertainty in Evidence

User goal
Understand where information is limited or inconclusive.

Description
The system flags areas where evidence is sparse, conflicting, or absent for selected supplement combinations.

System boundary

- Identifies uncertainty
- Does not speculate beyond available data

Outcome

- User sees where scientific knowledge is incomplete

UC-10. Re-analyze Stored Supplements Over Time

User goal
Revisit analysis as the personal library grows.

Description
The user selects previously stored supplements to perform new interaction analyses as additional supplements are added or as evidence data is updated.

System boundary

- Analysis uses updated data only
- No historical inference about user behavior

Outcome

- Ongoing access to updated evidence-based information

UC-11. Control Data Storage and Consent

User goal
Decide how their scanned data is stored and used.

Description
The user manages consent for storing scanned supplement data locally and for optional evidence updates.

System boundary

- No implicit data sharing
- Clear user control

Outcome

- User maintains ownership of personal supplement data

Use Case Scope Summary (Important)

Included

- Information retrieval
- Evidence aggregation
- Comparison
- Uncertainty exposure

Explicitly Excluded

- Safety judgments
- Dosage guidance
- Health recommendations
- Personalized advice

Coach's closing perspective (straight talk)

These use cases **prove alignment** between:

- Your architectural constraints
- Your AI boundaries
- Your stated research hypothesis

They are also **reviewer-safe**: every use case can be defended as *informational*, not advisory.

If you want next, I can:

- Turn these into **fully dressed use case specifications**
- Or map each use case to **specific architectural components and pipelines**

> for each use case, develop an interaction model in terms of components across layers int the architecture. The model should capture flow/process through sequential component interactions starting from the user initiation to the last component where the process ends throughout layers.

Below is a **systematic interaction model** for **each user-facing use case**, expressed as **sequential component interactions across architectural layers**.

I'll use a **consistent notation** so this can be lifted directly into:

- sequence diagrams,
- C4 dynamic views,

- or a paper appendix.

Notation

```scss
[User]
 → (Presentation)
 → (Application / Orchestration)
 → (AI Processing)
 → (Domain Logic)
 → (Data)
 → (Policy Enforcement)
 → (Presentation)
```

Only **allowed dependencies** are used.
Every flow **terminates at Presentation**, never inside AI or Domain logic.

UC-1. Scan a Supplement Label

User initiates: Scan with camera

Interaction Flow

```sql
User
 → CameraScanView
 → ScanWorkflowController
 → OCRService
 → IngredientExtractionService
 → IngredientNormalizationService
 → PolicyValidator
 → UserSupplementRepository
 → ScanResultViewModelBuilder
 → SupplementDetailView
```

Key Boundary Notes

- OCR & extraction return **verbatim text + normalized identifiers**
- PolicyValidator blocks any inferred meaning
- Stored as factual scanned record only

UC-2. Search for a Supplement by Name or Ingredient

User initiates: Text search

Interaction Flow

```sql
User
 → SearchView
 → SearchController
 → SupplementRepository
 → UserSupplementRepository
 → DomainMatchingEngine
 → PolicyValidator
 → SearchResultViewModelBuilder
 → SearchResultsView
```

Key Boundary Notes

- DomainMatchingEngine matches names/ingredients only
- No ranking by “quality” or “effectiveness”

UC-3. View Supplement Details

User initiates: Selects a supplement

Interaction Flow

```sql
User
 → SupplementListView
 → SupplementDetailController
 → SupplementRepository / UserSupplementRepository
 → DomainSupplementAssembler
 → PolicyValidator
 → SupplementDetailView
```

Key Boundary Notes

- No AI involved
- Pure data retrieval + formatting

UC-4. Build a Personal Supplement Library

User initiates: Save supplement

Interaction Flow

```sql
User
 → SaveToLibraryAction
 → LibraryController
 → PolicyConsentChecker
 → UserSupplementRepository
 → LibraryUpdateNotifier
 → LibraryView
```

Key Boundary Notes

- Explicit consent check
- No analysis triggered

UC-5. Select Multiple Supplements for Analysis

User initiates: Multi-select supplements

Interaction Flow

```sql
User
 → MultiSelectView
 → AnalysisSelectionController
 → SessionSelectionState
 → PolicyValidator
 → AnalysisReadyView
```

Key Boundary Notes

- No computation yet
- Only selection state updated

UC-6. View Documented Supplement Interactions

User initiates: Run interaction analysis

Interaction Flow

```scss
User
 → InteractionAnalysisView
 → AnalysisController
 → DomainInteractionEngine
 → EvidenceRepository
 → EvidencePatternExtractionService (AI)
 → EvidenceClassificationService (AI)
 → PolicyValidator
 → InteractionResultViewModelBuilder
 → InteractionEvidenceView
```

Key Boundary Notes

- AI only extracts and categorizes evidence
- DomainInteractionEngine defines *what interactions exist*, not *what they mean*

UC-7. Compare Supplements

User initiates: Compare selected supplements

Interaction Flow

```sql
User
 → ComparisonView
 → ComparisonController
 → SupplementRepository / UserSupplementRepository
 → DomainComparisonEngine
 → PolicyValidator
 → ComparisonTableViewModelBuilder
 → ComparisonResultsView
```

Key Boundary Notes

- No AI required
- Tabular comparison only (no preference logic)

UC-8. Review Evidence Sources and Strength

User initiates: Inspect evidence details

Interaction Flow

```sql
User
 → EvidenceDetailView
 → EvidenceDetailController
 → EvidenceRepository
 → DomainEvidenceAssembler
 → PolicyValidator
 → EvidenceSourceView
```

Key Boundary Notes

- Evidence strength shown as categories
- No confidence scoring

UC-9. Identify Gaps or Uncertainty in Evidence

User initiates: Toggle “uncertainty view”

Interaction Flow

```scss
User
 → UncertaintyToggleView
 → EvidenceAnalysisController
```

```
 → EvidenceRepository
 → EvidenceGapDetectionService (AI)
 → PolicyValidator
 → UncertaintySummaryViewModelBuilder
 → UncertaintyView
```

Key Boundary Notes

- AI flags absence/conflict only
- No speculative filling of gaps

UC-10. Re-analyze Stored Supplements Over Time

User initiates: Re-run analysis

Interaction Flow

```scss
User
 → LibraryAnalysisView
 → AnalysisController
 → UserSupplementRepository
 → DomainInteractionEngine
 → EvidenceRepository
 → EvidencePatternExtractionService (AI)
 → PolicyValidator
 → UpdatedAnalysisView
```

Key Boundary Notes

- Same pipeline as UC-6
- New data, same constraints

UC-11. Control Data Storage and Consent

User initiates: Manage settings

Interaction Flow

```sql
User
 → SettingsView
 → ConsentController
 → PolicyConsentManager
 → UserPreferencesStore
 → ConfirmationView
```

Key Boundary Notes

- No AI involved

- Direct policy enforcement

Cross-Use-Case Structural Invariants (Important)

✔️ User input always starts in **Presentation**
✔️ AI outputs always pass through **PolicyValidator**
✔️ No flow ends in AI or Domain layers
✔️ No upward call from Data → Domain → AI

Why this interaction model matters (straight answer)

This makes your claim **provable**:

> *NutriCompass cannot make decisions because no interaction path allows it.*

Every use case:

- ends in **information presentation**
- never transitions into advice
- never fuses user context with inference

If you want next, I can:

- Convert one or more of these into **formal UML sequence diagrams**
- Or compress this into a **paper-ready "Interaction Model" section**

> What graphical notation would be suitable to capture the layers of the architecture?

To capture **architectural layers** clearly, defensibly, and in a way that scales from *design* to *paper* to *implementation*, you want a notation that makes **boundaries, allowed dependencies, and flow direction** explicit.

Below is a **practical recommendation stack**, not a single answer—because different notations serve *different explanatory goals*. I'll be direct about **what to use, when, and why**.

Primary Recommendation (Best Fit)

C4 Model – Container + Component Diagrams

Why C4 is the best primary notation for NutriCompass

C4 diagrams are:

- Layer-friendly
- Boundary-explicit
- Reviewer-readable (non-UML experts can follow)
- Excellent for **"decision support, not decision making" arguments**

How to use C4 for your layers

1️⃣ C4 Container Diagram (Layer Overview)

Use containers to represent **layers**:

```pgsql

+---------------------------------------+
| Presentation Container                |
+---------------------------------------+
| Application / Orchestration Container |
+---------------------------------------+
| Domain Logic Container                |
+---------------------------------------+
| AI Processing Container               |
+---------------------------------------+
| Data Container                        |
+---------------------------------------+
| Policy & Enforcement Container        |
+---------------------------------------+
```

- Arrows show **allowed dependency direction**
- No arrows upward
- Policy container has "observe / validate" links

This diagram answers:

> *"Where does AI live and what can it touch?"*

2️⃣ C4 Component Diagrams (Inside Each Layer)

Within each container:

- Components = controllers, services, repositories
- Dependencies are explicit and directional

Example inside **AI Processing Container**:

- OCRService
- IngredientExtractionService
- EvidenceClassificationService
- OutputSchemaValidator

This shows:

- AI is modular
- AI has no UI or decision logic

Secondary Recommendation (Formal Precision)

UML Package Diagrams (Layered Packages)

Why UML Package Diagrams help

They:

- Explicitly encode **layer boundaries**
- Allow **dependency constraints**
- Are useful in academic or standards contexts

How to model layers

```pgsql
<<package>> Presentation
<<package>> Application
<<package>> Domain
<<package>> AI
<<package>> Data
<<package>> Policy
```

- Dependencies allowed only top → bottom
- Use UML dependency arrows
- Add stereotypes like `<<no_decision_logic>>`

This answers:

> *"What is allowed to depend on what?"*

Flow Modeling (Complementary, Not for Layers)

UML Sequence Diagrams

Use these **after** layer diagrams.

- One sequence per use case
- Components are grouped visually by layer
- Reinforces that flow crosses layers but authority does not

This answers:

> *"How does information move without turning into advice?"*

Optional / Research-Oriented

Layered Architecture Diagram (ISO/IEC 42010 Style)

For academic papers:

- Use a **stacked layer diagram**
- Annotate each layer with:
  - Allowed responsibilities
  - Explicit prohibitions

Good for:

- Architecture sections
- Design philosophy justification

What NOT to Use (Important)

❌ **Class diagrams alone**

- Too granular
- Hide architectural intent

❌ **Free-form block diagrams**

- Look hand-wavy
- Hard to defend boundaries

❌ **Only sequence diagrams**

- Show flow, not structure

Recommended Combination (Strongest)

Purpose	Notation
Architectural layers	**C4 Container Diagram**
Internal structure	**C4 Component Diagrams**
Dependency constraints	**UML Package Diagram**
Use case flow	**UML Sequence Diagrams**

This combo lets you:

- Explain the system clearly

- Prove boundary enforcement
- Scale from slides → paper → implementation

Coach's bottom line (plainspoken)

If NutriCompass is making an *architectural argument*—not just building software—**C4 + UML Package** is the cleanest, most defensible choice.

If you want, next I can:

- Sketch a **C4 container diagram textually**
- Or translate one use case into a **layer-grouped sequence diagram**

> develop the internal structure of each layer in terms of its composing components using C4 component diagrams.

Below is a **C4-style Component breakdown for *each architectural layer*** of NutriCompass.
This is written so it can be **directly converted into C4 Component Diagrams** (Structurizr, Mermaid, PlantUML, diagrams.net), and it makes **boundaries and non-dependencies explicit**.

I'll follow a **consistent structure per layer**:

- Layer intent (1 sentence)
- C4 Container context
- Internal components (with responsibilities)
- Allowed internal dependencies
- Explicitly forbidden dependencies (this is important)

---

1. Presentation Layer – C4 Component Diagram

Intent
Render structured information and evidence without interpretation or inference.

C4 Container

Container: `Presentation UI`

Components

- **ScanView**
  - Camera capture UI
  - User-initiated scanning only

- **SearchView**
  - Text-based search input
- **SupplementListView**
  - Displays database + library items
- **SupplementDetailView**
  - Displays ingredient and formulation facts
- **InteractionEvidenceView**
  - Displays interaction summaries and evidence cards
- **ComparisonView**
  - Side-by-side factual comparison
- **EvidenceDetailView**
  - Source-level evidence inspection
- **SettingsView**
  - Consent and storage controls
- **ViewModelRenderer**
  - Renders *validated* view models only

Dependencies

```pgsql

Views → Application Controllers
Views → ViewModelRenderer
ViewModelRenderer → Policy Validator (read-only)
```

Forbidden

- ❌ Presentation → AI
- ❌ Presentation → Data Repositories
- ❌ Presentation → Domain Models

---

2. Application / Orchestration Layer – C4 Component Diagram

Intent
Coordinate workflows and route requests without intelligence or judgment.

C4 Container

Container: `Application Services`

Components

- **ScanWorkflowController**
  - Orchestrates scan pipeline
- **SearchController**
  - Coordinates search queries
- **LibraryController**
  - Handles save/remove actions
- **AnalysisController**
  - Coordinates interaction analysis
- **ComparisonController**
  - Coordinates comparison workflows
- **EvidenceDetailController**
  - Retrieves evidence details
- **SessionSelectionState**
  - Tracks user-selected supplements
- **ViewModelBuilder**
  - Assembles presentation-safe models

Dependencies

```pgsql
Controllers → Domain Logic
Controllers → AI Processing (via interfaces)
Controllers → Policy Validator
Controllers → Repositories (interfaces only)
ViewModelBuilder → Policy Validator
```

Forbidden

- ❌ Controllers → AI internals
- ❌ Controllers → Raw data storage
- ❌ Controllers → UI logic

---

3. Domain Logic Layer – C4 Component Diagram

Intent
Define entities, relationships, and deterministic rules without inference.

C4 Container

Container: `Domain Model`

Components

- **Supplement**
  - Canonical supplement entity
- **Ingredient**
  - Active compound representation
- **Interaction**
  - Defined interaction relationship
- **EvidenceRecord**
  - Link between interaction and sources
- **InteractionTaxonomy**
  - Absorption, metabolism, antagonism, etc.
- **DomainInteractionEngine**
  - Identifies applicable interactions
- **DomainComparisonEngine**
  - Aligns ingredients across supplements
- **DomainAssembler**
  - Builds domain objects from repositories

Dependencies

```pgsql
Domain Engines → Repositories (interfaces)
Domain Engines → InteractionTaxonomy
Domain Assemblers → Domain Entities
```

Forbidden

- ❌ Domain → AI
- ❌ Domain → Presentation
- ❌ Domain → User context or preferences

---

4. AI Processing Layer – C4 Component Diagram

Intent
Extract, normalize, and classify factual information only.

C4 Container

Container: `AI Processing Services`

Components

- **OCRService**
  - Extracts raw text from images
- **IngredientExtractionService**
  - Identifies ingredient mentions
- **NormalizationService**
  - Resolves synonyms and identifiers
- **LiteratureRetrievalService**
  - Fetches relevant publications
- **EvidencePatternExtractor**
  - Detects reported interactions
- **EvidenceClassificationService**
  - Tags evidence strength and conflict
- **UncertaintyDetectionService**
  - Flags gaps or inconsistencies
- **AIOutputSchemaValidator**
  - Enforces allowed output structures

Dependencies

```pgsql

AI Services → Data Layer (read-only)
AI Services → AIOutputSchemaValidator
AI Services → Policy Rules
```

Forbidden

- ❌ AI → Domain rules
- ❌ AI → User data
- ❌ AI → UI language
- ❌ AI → Recommendation logic

---

5. Data Layer – C4 Component Diagram

Intent
Store and retrieve data without behavior or interpretation.

C4 Container

Container: `Data Stores`

Components

- **SupplementRepository**
  - Pre-loaded supplement data
- **UserSupplementRepository**
  - User-scanned supplements
- **EvidenceRepository**
  - Scientific studies and reports
- **IngredientIndex**
  - Fast ingredient lookup
- **SynonymIndex**
  - Ingredient name resolution
- **ProvenanceStore**
  - Source and version tracking

Dependencies

```
nginx


Repositories → Storage Engine
Indexes → Repositories
```

Forbidden

- ❌ Data → AI
- ❌ Data → Domain Logic
- ❌ Data → Application Controllers

---

6. Policy & Boundary Enforcement Layer – C4 Component Diagram

Intent
Guarantee “decision support, not decision making”.

C4 Container

Container: `Policy & Enforcement`

Components

- **LanguagePolicyEngine**
  - Allowed / prohibited phrasing
- **SchemaPolicyEngine**
  - Validates structural outputs

- **ContextFirewall**
  - Blocks access to user health context
- **ConsentPolicyManager**
  - Enforces storage permissions
- **AuditLogger**
  - Logs AI inputs/outputs
- **ViolationBlocker**
  - Stops propagation of invalid data

Dependencies

```pgsql
Policy Engines ← Observe all layers
ViolationBlocker → Application Controllers
AuditLogger → Secure Storage
```

Forbidden

- ❌ Policy → Business logic
- ❌ Policy → AI inference

---

7. Infrastructure Layer – C4 Component Diagram

Intent
Provide execution, isolation, and security.

C4 Container

Container: `Platform Infrastructure`

Components

- **OnDeviceMLRuntime**
- **SecureLocalStorage**
- **OptionalCloudSyncService**
- **ModelVersionManager**
- **EncryptionService**

Dependencies

```scss
All layers → Infrastructure (indirect)
```

Cross-Layer C4 Integrity Rules (Critical)

✔️ Components only depend **downward or sideways within a container**
✔️ AI components are isolated behind interfaces
✔️ Policy components can **block but never infer**
✔️ No component has both **AI + decision authority**

Why this C4 structure is strong (straight answer)

This gives you:

- Clear **component ownership**
- Enforceable **non-dependencies**
- A diagram set that *proves* your system philosophy

It's not just descriptive—it's **constraining**.

If you want next

I can:

1. Convert one layer into **PlantUML / Mermaid syntax**
2. Build a **single consolidated C4 diagram set**
3. Map these components directly to **Swift modules for NutriCompass**

Just tell me which.

> Develop UML package diagram capturing the dependency constraints among components across layers.

```
plantuml


@startuml
title NutriCompass — UML Package Diagram (Layered Dependency Constraints)
skinparam packageStyle rectangle
skinparam shadowing false
skinparam linetype ortho

' ==========================
' LAYERS AS PACKAGES
' ==========================

package "Presentation Layer\n<<UI>>" as P {
  [ScanView]
  [SearchView]
  [LibraryView]
  [ComparisonView]
  [InteractionEvidenceView]
```

```
  [EvidenceDetailView]
  [SettingsView]
  [ViewModelRenderer]
}

package "Application / Orchestration Layer\n<<AppSvc>>" as A {
  [ScanWorkflowController]
  [SearchController]
  [LibraryController]
  [AnalysisController]
  [ComparisonController]
  [EvidenceDetailController]
  [SessionSelectionState]
  [ViewModelBuilder]
}

package "Domain Logic Layer\n<<Domain>>" as D {
  [Supplement]
  [Ingredient]
  [Interaction]
  [EvidenceRecord]
  [InteractionTaxonomy]
  [DomainAssembler]
  [DomainMatchingEngine]
  [DomainComparisonEngine]
  [DomainInteractionEngine]
}

package "AI Processing Layer\n<<AI>>" as I {
  [OCRService]
  [IngredientExtractionService]
  [NormalizationService]
  [LiteratureRetrievalService]
  [EvidencePatternExtractor]
  [EvidenceClassificationService]
  [UncertaintyDetectionService]
  [AIOutputSchemaValidator]
}

package "Data Layer\n<<Data>>" as R {
  [SupplementRepository]
  [UserSupplementRepository]
  [EvidenceRepository]
  [IngredientIndex]
  [SynonymIndex]
  [ProvenanceStore]
}

package "Policy & Enforcement\n<<CrossCutting>>" as X {
  [LanguagePolicyEngine]
  [SchemaPolicyEngine]
  [ContextFirewall]
  [ConsentPolicyManager]
  [AuditLogger]
  [ViolationBlocker]
}

package "Infrastructure Layer\n<<Platform>>" as F {
  [OnDeviceMLRuntime]
  [SecureLocalStorage]
  [ModelVersionManager]
  [EncryptionService]
  [OptionalCloudSyncService]
}

' ==========================
' ALLOWED DEPENDENCIES (HARD CONSTRAINTS)
' ==========================

' Presentation may depend ONLY on Application (+ read-only policy for validation-aware
rendering)
P ..> A : UI events / ViewModels
P ..> X : read-only validation gates\n(phrasing/templates)

' Application orchestrates Domain + AI via interfaces and uses repositories via interfaces
```

```
A ..> D : use domain services/entities
A ..> I : call AI services (interfaces)
A ..> R : repositories (interfaces only)
A ..> X : validate, consent, block, audit

' Domain uses repositories (interfaces). Domain does NOT depend on AI or UI.
D ..> R : read models via repos
D ..> X : invariant validation\n(taxonomy/schema checks)

' AI uses data (read-only) and policy (schema/language constraints)
I ..> R : read evidence / indices
I ..> X : output constraints\n(schema + prohibited language)
I ..> F : runtime + model mgmt

' Data uses only infrastructure
R ..> F : persistence / indexing

' Policy uses infra (logging/secure storage) and may observe all layers (modeled as dependencies
here)
X ..> F : secure logging / policy storage

' ==========================
' OPTIONAL: "OBSERVES" LINKS (NON-FUNCTIONAL, FOR AUDIT/VALIDATION)
' ==========================
X ..> P : observe outputs
X ..> A : observe orchestration
X ..> D : observe domain objects
X ..> I : observe AI IO
X ..> R : observe data access

' ==========================
' FORBIDDEN DEPENDENCIES (DOCUMENTED CONSTRAINTS)
' ==========================
note "Forbidden dependencies (must NOT exist):\n\n1) Presentation -> AI, Presentation -> Data,
Presentation -> Domain\n2) Domain -> AI, Domain -> Presentation, Domain -> Application\n3) AI ->
Presentation, AI -> Domain, AI -> User context\n4) Data -> (any of:
AI/Domain/App/Presentation)\n5) Any bypass of Policy validation in flows that reach
Presentation\n\nRationale: enforce 'decision support, not decision making' and prevent AI
authority leakage." as N1

N1 .. P
N1 .. A
N1 .. D
N1 .. I
N1 .. R
N1 .. X

' ==========================
' ADDITIONAL CONSTRAINTS (HELPFUL FOR REVIEWERS)
' ==========================
note "Hard-rail constraints:\n- All AI outputs MUST pass SchemaPolicyEngine +
LanguagePolicyEngine via ViolationBlocker.\n- ContextFirewall prevents AI access to user
health/safety context.\n- ViewModelRenderer renders only validated ViewModels.\n- Repositories
expose read/write via interfaces; concrete storage hidden in Infrastructure." as N2

N2 .. X

@enduml
```

How to read this diagram (quick)

- **Allowed dependencies** are modeled as `..>` arrows.
- The **Policy package** is modeled as cross-cutting: it can *observe/validate/block* across layers, but it must not perform domain decisions or AI inference.
- The **Forbidden dependency list** is included as a UML note so reviewers can immediately see the architectural constraints you enforce.

If you want, I can also generate a **second version** that uses **only package-to-package arrows** (no internal components) for a cleaner “one-slide” diagram, while keeping the same constraints.

develop UML sequence diagrams capturing use case flow in terms of components across layers.

Below are **UML sequence diagrams (PlantUML)** that capture **use case flows across architectural layers**, using **the exact components you defined earlier** and **respecting all dependency constraints**.

To keep this rigorous *and usable*, I provide **five representative diagrams** that together cover **all interaction patterns**.
(Several use cases are structural variants of these.)

You can:

- paste these directly into PlantUML / Structurizr,
- include them in an appendix,
- or convert them to figures for a paper.

UC-1 — Scan a Supplement Label

Purpose: Demonstrates *AI extraction without interpretation* and persistence into user library.

```
plantuml


@startuml
title UC-1: Scan Supplement Label

actor User

participant "ScanView\n(Presentation)" as UI
participant "ScanWorkflowController\n(Application)" as App
participant "OCRService\n(AI)" as OCR
participant "IngredientExtractionService\n(AI)" as IE
participant "NormalizationService\n(AI)" as Norm
participant "SchemaPolicyEngine\n(Policy)" as Policy
participant "UserSupplementRepository\n(Data)" as Repo
participant "SupplementDetailView\n(Presentation)" as View

User -> UI : capture label image
UI -> App : startScan(image)

App -> OCR : extractText(image)
OCR --> App : rawText

App -> IE : extractIngredients(rawText)
IE --> App : ingredientMentions

App -> Norm : normalize(ingredientMentions)
Norm --> App : normalizedIngredients

App -> Policy : validateSchema(normalizedIngredients)
Policy --> App : OK

App -> Repo : saveScannedSupplement(normalizedIngredients)
Repo --> App : supplementId

App -> View : buildDetailViewModel(supplementId)
```

```
View --> User : display extracted supplement info
@enduml
```

Key architectural proof

- AI never reaches UI
- Policy gate before storage and presentation
- No recommendation path exists

UC-2 — Search for Supplement (Database + Library)

```
plantuml


@startuml
title UC-2: Search Supplement

actor User

participant "SearchView\n(Presentation)" as UI
participant "SearchController\n(Application)" as App
participant "SupplementRepository\n(Data)" as DB
participant "UserSupplementRepository\n(Data)" as Lib
participant "DomainMatchingEngine\n(Domain)" as Match
participant "SchemaPolicyEngine\n(Policy)" as Policy
participant "SearchResultsView\n(Presentation)" as View

User -> UI : enter query
UI -> App : search(query)

App -> DB : findByNameOrIngredient(query)
DB --> App : dbResults

App -> Lib : findByNameOrIngredient(query)
Lib --> App : userResults

App -> Match : mergeAndMatch(dbResults, userResults)
Match --> App : matchedResults

App -> Policy : validateOutput(matchedResults)
Policy --> App : OK

App -> View : presentResults(matchedResults)
View --> User : show supplement list
@enduml
```

Key architectural proof

- Domain handles matching, not AI
- No ranking by “quality” or “effectiveness”

UC-5 & UC-6 — Select Supplements + View Interactions

(Core analytical use case)

```
plantuml


@startuml
title UC-6: Analyze Documented Supplement Interactions
```

```
actor User

participant "InteractionAnalysisView\n(Presentation)" as UI
participant "AnalysisController\n(Application)" as App
participant "DomainInteractionEngine\n(Domain)" as Domain
participant "EvidenceRepository\n(Data)" as EvidenceDB
participant "EvidencePatternExtractor\n(AI)" as AI
participant "EvidenceClassificationService\n(AI)" as Classifier
participant "SchemaPolicyEngine\n(Policy)" as Policy
participant "InteractionEvidenceView\n(Presentation)" as View

User -> UI : analyze selected supplements
UI -> App : runAnalysis(supplementIds)

App -> Domain : identifyInteractions(supplementIds)
Domain -> EvidenceDB : fetchEvidence(interactions)
EvidenceDB --> Domain : evidenceRecords
Domain --> App : interactionCandidates

App -> AI : extractPatterns(evidenceRecords)
AI --> App : evidencePatterns

App -> Classifier : classifyEvidence(evidencePatterns)
Classifier --> App : classifiedEvidence

App -> Policy : validateSchema(classifiedEvidence)
Policy --> App : OK

App -> View : buildEvidenceViewModel(classifiedEvidence)
View --> User : display documented interactions
@enduml
```

Key architectural proof

- Domain defines *what interactions exist*
- AI only structures evidence
- Policy blocks interpretive language

UC-7 — Compare Supplements

```
plantuml


@startuml
title UC-7: Compare Supplements

actor User

participant "ComparisonView\n(Presentation)" as UI
participant "ComparisonController\n(Application)" as App
participant "SupplementRepository\n(Data)" as DB
participant "UserSupplementRepository\n(Data)" as Lib
participant "DomainComparisonEngine\n(Domain)" as Domain
participant "SchemaPolicyEngine\n(Policy)" as Policy
participant "ComparisonResultsView\n(Presentation)" as View

User -> UI : compare supplements
UI -> App : compare(ids)

App -> DB : fetchSupplements(ids)
DB --> App : dbSupplements

App -> Lib : fetchSupplements(ids)
Lib --> App : userSupplements

App -> Domain : alignAndCompare(dbSupplements, userSupplements)
```

```
Domain --> App : comparisonResult

App -> Policy : validateOutput(comparisonResult)
Policy --> App : OK

App -> View : presentComparison(comparisonResult)
View --> User : show comparison table
@enduml
```

Key architectural proof

- No AI required
- No “better than” inference path exists

UC-9 — Identify Evidence Gaps / Uncertainty

```
plantuml


@startuml
title UC-9: Identify Evidence Gaps

actor User

participant "UncertaintyView\n(Presentation)" as UI
participant "AnalysisController\n(Application)" as App
participant "EvidenceRepository\n(Data)" as EvidenceDB
participant "UncertaintyDetectionService\n(AI)" as AI
participant "SchemaPolicyEngine\n(Policy)" as Policy
participant "UncertaintySummaryView\n(Presentation)" as View

User -> UI : view uncertainty
UI -> App : detectGaps(supplementIds)

App -> EvidenceDB : fetchEvidence(supplementIds)
EvidenceDB --> App : evidenceRecords

App -> AI : detectGaps(evidenceRecords)
AI --> App : uncertaintyFlags

App -> Policy : validateSchema(uncertaintyFlags)
Policy --> App : OK

App -> View : presentUncertainty(uncertaintyFlags)
View --> User : show gaps/conflicts
@enduml
```

Key architectural proof

- AI flags *absence and conflict only*
- No speculative completion of missing data

Coverage Mapping (Completeness)

Use Case	Covered by Diagram
UC-1 Scan	Diagram 1
UC-2 Search	Diagram 2
UC-3 View Details	UC-1 / UC-2 tail

Use Case	Covered by Diagram
UC-4 Library	UC-1 storage path
UC-5 Select	UC-6 setup
UC-6 Interactions	Diagram 3
UC-7 Compare	Diagram 4
UC-8 Evidence detail	UC-6 extension
UC-9 Uncertainty	Diagram 5
UC-10 Re-analysis	UC-6 repeat
UC-11 Consent	trivial sequence (omitted for brevity)

Why these sequence diagrams are strong (straight talk)

They **prove by construction** that:

- AI cannot reach users directly
- Every AI output is policy-gated
- No path exists to recommendations, judgments, or advice

This is not just documentation — it's **architectural evidence**.

If you want next

I can:

1. Compress these into **one consolidated "analysis pipeline" diagram**
2. Produce **publication-ready vector figures**
3. Map each sequence to **Swift execution paths**

Just say the word.